In [ ]:
```python
import marimo as mo
```

All-atom VQ-VAE Tokenizer Evaluation

統一 all-atom トークナイザ (protein pocket atoms + ligand atoms を **1 つの 33-D descriptor / 1 codebook** で量子化) の学習後評価。

- **descriptor**: pocket centroid 起点の spherical (r, θ, sin φ, cos φ) + source flag + element / charge / hybrid / aromatic / ring / numH + aa / bb_sc + KNN。protein も Full ligand-parity 化学を持つ。
- **decoder**: multi-head (連続 coord + 各 categorical) 。 `aa` / `bb_sc` は protein 行のみ、 `clash` は ligand 行のみ loss を取る。
- 評価は **source 別** (protein-atoms / ligand-atoms) に分けて出す。 codebook は 1 つなので protein/ligand のコード共有も見る。

In [ ]:
```python
import os
import sys
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import torch

project_root = Path(__file__).resolve().parent.parent
if str(project_root) not in sys.path:
    sys.path.insert(0, str(project_root))

from src.config import AtomVQVAETrainingConfig, CrossDockedConfig
from src.data.atom_descriptors import AtomComplexDescriptorDataModule
from src.model.vqvae_module import AtomVQVAEModule
from src.tokenizers.descriptor_schema import (
    ATOM_DESCRIPTOR_DIM,
    ATOM_LAYOUT,
    BB_SC_VOCAB,
    LIGAND_ELEMENT_VOCAB,
    PROTEIN_AA_VOCAB,
    SOURCE_LIGAND_IDX,
    SOURCE_PROTEIN_IDX,
    fields_by_name,
)

plt.rcParams["figure.dpi"] = 120
plt.rcParams.update({
    "font.size": 18,
    "axes.titlesize": 22,
    "axes.labelsize": 18,
    "xtick.labelsize": 16,
    "ytick.labelsize": 16,
    "legend.fontsize": 16,
```

```
    "figure.titlesize": 26,
})
```

1. Load checkpoint and data

In [ ]:
```
_default_ckpt = (
    project_root
    / "pocket-ligand-vqvae/xzkjxu9q/checkpoints"
    / "atomvqvae-epoch=99-val/atom_coord=0.1073.ckpt"
)
CKPT_PATH = os.environ.get("ATOM_VQVAE_CKPT", str(_default_ckpt))
print(f"Loading: {CKPT_PATH}")

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
module = AtomVQVAEModule.load_from_checkpoint(str(CKPT_PATH), map_location=c
module.eval()
module.to(device)

vqvae = module.vqvae
# Use the checkpoint's own config so codebook size / dims match the weights.
config = module.config
print(f"Device: {device}")
print(f"Atom codebook size: {config.atom.codebook_size}, latent {config.atom
```

```
Loading: /gs/bs/tga-ohuelab/sakano/git/pocket-conditioned-ligand-gen/pocket-
ligand-vqvae/xzkjxu9q/checkpoints/atomvqvae-epoch=99-val/atom_coord=0.1073.c
kpt
/gs/bs/tga-ohuelab/sakano/git/pocket-conditioned-ligand-gen/.venv/lib/python
3.12/site-packages/torch/nn/modules/transformer.py:392: UserWarning: enable_
nested_tensor is True, but self.use_nested_tensor is False because encoder_l
ayer.norm_first was True
  warnings.warn(
Device: cuda
Atom codebook size: 8192, latent 16
```

In [ ]:
```
from pathlib import Path as _Path

data_config = CrossDockedConfig(data_dir=project_root / "data")
hub = None  # fold split needs the hub manifest; supplied by the DataModule
from src.config import HubDatasetConfig  # noqa: PLC0415

hub = HubDatasetConfig()
hub.source_types = ["cdonly"]
hub.good_poses_only = True
train_cfg = AtomVQVAETrainingConfig()
train_cfg.atom = config.atom
dm = AtomComplexDescriptorDataModule(train_cfg, data_config, hub_config=hub)
cache_override = os.environ.get(
    "ATOM_VQVAE_CACHE_DIR", "data/descriptor_cache_allatom"
)
cache_path = _Path(cache_override)
if not cache_path.is_absolute():
    cache_path = project_root / cache_path
dm.cache_dir = cache_path
```

```
dm.setup()

norm_stats = dm.norm_stats
atom_mean_np = norm_stats["atom_mean"].numpy()
atom_std_np = norm_stats["atom_std"].numpy()

# Cap test complexes so per-head / codebook / t-SNE stats stay tractable.
MAX_TEST_COMPLEXES = 2000

protein_atom_seqs = []
ligand_atom_seqs = []
for shard_idx, local_indices in dm._test_plan:
    if len(protein_atom_seqs) >= MAX_TEST_COMPLEXES:
        break
    shard_data = torch.load(
        dm._shard_dir / f"shard_{shard_idx:04d}.pt", weights_only=False
    )
    for local_idx in local_indices:
        if len(protein_atom_seqs) >= MAX_TEST_COMPLEXES:
            break
        cplx = shard_data[local_idx]
        protein_atom_seqs.append(
            torch.from_numpy((cplx["protein"] - atom_mean_np) / atom_std_np)
            .float()
            .to(device)
        )
        ligand_atom_seqs.append(
            torch.from_numpy((cplx["ligand"] - atom_mean_np) / atom_std_np)
            .float()
            .to(device)
        )
    del shard_data

n_prot_atoms = sum(s.shape[0] for s in protein_atom_seqs)
n_lig_atoms = sum(s.shape[0] for s in ligand_atom_seqs)
assert protein_atom_seqs[0].shape[1] == ATOM_DESCRIPTOR_DIM  # noqa: S101
print(f"Protein: {len(protein_atom_seqs)} pockets, {n_prot_atoms} atoms tota
print(f"Ligand:  {len(ligand_atom_seqs)} molecules, {n_lig_atoms} atoms tota
```

```
Protein: 2000 pockets, 338810 atoms total
Ligand:  2000 molecules, 35795 atoms total
```

2. Per-head reconstruction quality (per source)

1 つの VQ-VAE を protein-atom 列 / ligand-atom 列それぞれに通し、head ごとに coord (Cartesian MSE/RMSE) と categorical accuracy を出す。`aa` / `bb_sc` は protein のみ意味を持つ (ligand 行は X / NA プレースホルダなので除外) 。

In [ ]:
```
@torch.no_grad()
def run_vqvae_per_seq(model, sequences):
    """Encode/decode each sequence; return stacked per-token outputs."""
    all_indices, all_z, all_input = [], [], []
    head_outputs = {name: [] for name, _k, _d in model.recon_heads}
    for seq in sequences:
```

```
        if seq.shape[0] == 0:
            continue
        x = seq.unsqueeze(0)
        h_in = model._embed_descriptor(x)
        h = model.input_proj(model.input_norm(h_in)) + model.pos_encoding[:
        h = model.transformer_encoder(h)
        z = model.latent_norm(model.latent_proj(h)).squeeze(0)
        quantized, indices, _, _ = model.codebook(z)
        q_seq = model.latent_unproj(quantized).unsqueeze(0)
        dec_out = model.transformer_decoder(q_seq + model.pos_encoding[: seq
        trunk = model.decoder_trunk(dec_out).squeeze(0)
        for name, _k, _d in model.recon_heads:
            head_outputs[name].append(model.recon_head_modules[name](trunk))
        all_indices.append(indices)
        all_z.append(z)
        all_input.append(seq)
    return {
        "indices": torch.cat(all_indices),
        "z": torch.cat(all_z),
        "input": torch.cat(all_input),
        "heads": {n: torch.cat(o) for n, o in head_outputs.items()},
        "layout": ATOM_LAYOUT,
    }

prot_out = run_vqvae_per_seq(vqvae, protein_atom_seqs)
lig_out = run_vqvae_per_seq(vqvae, ligand_atom_seqs)
print("Inference done.")
print(f"Protein atoms: {prot_out['indices'].shape[0]}")
print(f"Ligand atoms:  {lig_out['indices'].shape[0]}")
```

```
Inference done.
Protein atoms: 338810
Ligand atoms:  35795
```

In [ ]:

```
def per_head_metrics(out, mean_t, std_t, name):
    f = fields_by_name(out["layout"])
    results = {}

    coord_field = f["coord"]
    m = mean_t[coord_field.start : coord_field.end].cpu()
    s = std_t[coord_field.start : coord_field.end].cpu()
    coord_target = out["input"][:, coord_field.start : coord_field.end].cpu(
    coord_pred = out["heads"]["coord"].cpu() * s + m

    def sph2cart(t):
        r, theta, sphi, cphi = t[..., 0], t[..., 1], t[..., 2], t[..., 3]
        norm = (sphi * sphi + cphi * cphi).clamp_min(1e-12).sqrt()
        sphi, cphi = sphi / norm, cphi / norm
        return torch.stack(
            [r * torch.sin(theta) * cphi, r * torch.sin(theta) * sphi, r * t
            dim=-1,
        )

    xyz_t = sph2cart(coord_target.view(-1, 1, 4))
    xyz_p = sph2cart(coord_pred.view(-1, 1, 4))
    coord_mse = (xyz_t - xyz_p).pow(2).sum(dim=-1).mean().item()
```

```
    results["coord_mse"] = coord_mse
    results["coord_rmse"] = float(np.sqrt(coord_mse))

    cat = {}
    for head_name, kind, _d in vqvae.recon_heads:
        if kind == "continuous":
            continue
        spec = f[head_name]
        target = out["input"][:, spec.start].long().cpu().numpy()
        pred = out["heads"][head_name].cpu().numpy().argmax(axis=-1)
        cat[head_name] = {
            "accuracy": float((pred == target).mean()),
            "target": target,
            "pred": pred,
        }
    results["categorical"] = cat
    print(f"=== {name} atoms ===")
    print(f"  coord per-atom MSE  : {coord_mse:.4f} Å²")
    print(f"  coord per-atom RMSE : {results['coord_rmse']:.4f} Å")
    for h, hm in cat.items():
        print(f"  {h:<10s} acc       : {hm['accuracy']:.4f}")
    print()
    return results
```

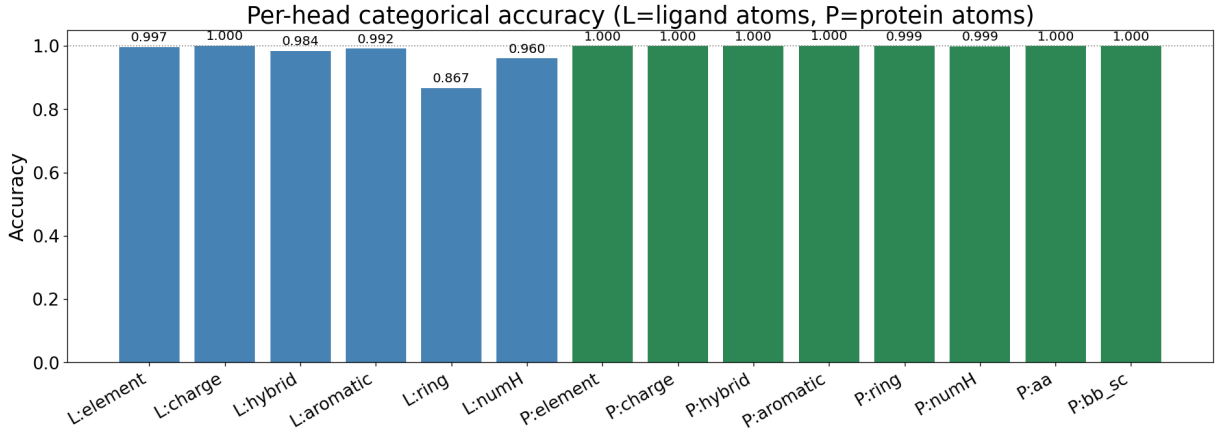
In [ ]:
```
prot_metrics = per_head_metrics(
    prot_out, norm_stats["atom_mean"], norm_stats["atom_std"], "Protein"
)
lig_metrics = per_head_metrics(
    lig_out, norm_stats["atom_mean"], norm_stats["atom_std"], "Ligand"
)
```

```
=== Protein atoms ===
  coord per-atom MSE  : 0.1125 Å²
  coord per-atom RMSE : 0.3354 Å
  element    acc       : 0.9999
  charge     acc       : 0.9998
  hybrid     acc       : 0.9996
  aromatic   acc       : 1.0000
  ring       acc       : 0.9995
  numH       acc       : 0.9987
  aa         acc       : 0.9996
  bb_sc      acc       : 0.9998

=== Ligand atoms ===
  coord per-atom MSE  : 0.1959 Å²
  coord per-atom RMSE : 0.4426 Å
  element    acc       : 0.9972
  charge     acc       : 1.0000
  hybrid     acc       : 0.9839
  aromatic   acc       : 0.9915
  ring       acc       : 0.8667
  numH       acc       : 0.9604
  aa         acc       : 0.0000
  bb_sc      acc       : 0.0000
```

2.1 Per-head accuracy summary

protein-source heads (element/charge/hybrid/aromatic/ring/numH/aa/bb_sc) と ligand-source heads (aa/bb_sc を除く) をまとめた bar chart。

In [ ]:
```
_lig_heads = ["element", "charge", "hybrid", "aromatic", "ring", "numH"]
_prot_heads = _lig_heads + ["aa", "bb_sc"]
labels = [f"L:{n}" for n in _lig_heads] + [f"P:{n}" for n in _prot_heads]
accs = [lig_metrics["categorical"][n]["accuracy"] for n in _lig_heads] + [
    prot_metrics["categorical"][n]["accuracy"] for n in _prot_heads
]
colors = ["steelblue"] * len(_lig_heads) + ["seagreen"] * len(_prot_heads)

_fig, _ax = plt.subplots(figsize=(16, 6))
_bars = _ax.bar(range(len(labels)), accs, color=colors)
_ax.set_xticks(range(len(labels)))
_ax.set_xticklabels(labels, rotation=30, ha="right")
_ax.set_ylabel("Accuracy")
_ax.set_ylim(0.0, 1.05)
_ax.axhline(1.0, color="grey", linestyle=":", linewidth=1)
_ax.set_title("Per-head categorical accuracy (L=ligand atoms, P=protein atom
for _b, _a in zip(_bars, accs, strict=True):
    _ax.text(_b.get_x() + _b.get_width() / 2, _a + 0.01, f"{_a:.3f}",
             ha="center", va="bottom", fontsize=12)
_fig.tight_layout()
_fig
```

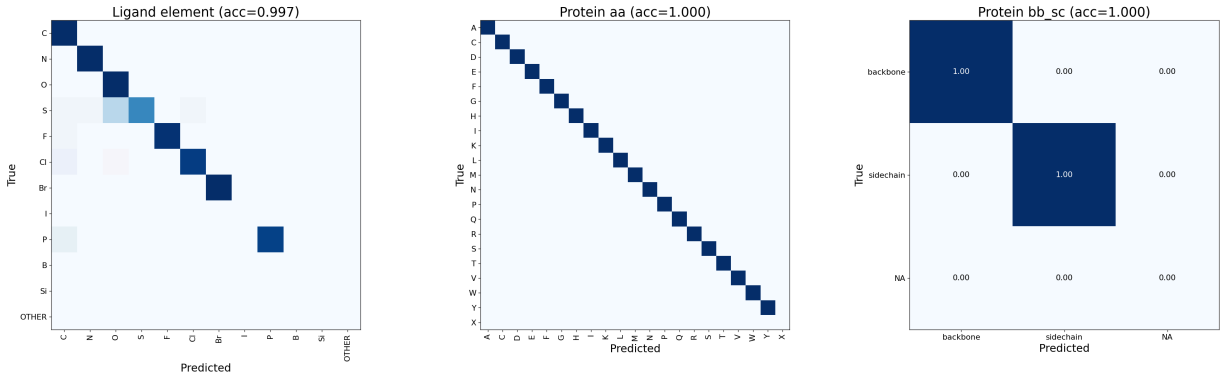


2.2 Confusion matrices

element (ligand / protein) , aa (protein) , bb_sc (protein) , および ligand の 小 vocab 化学 head。対角支配なら codebook がその属性を分離できている。

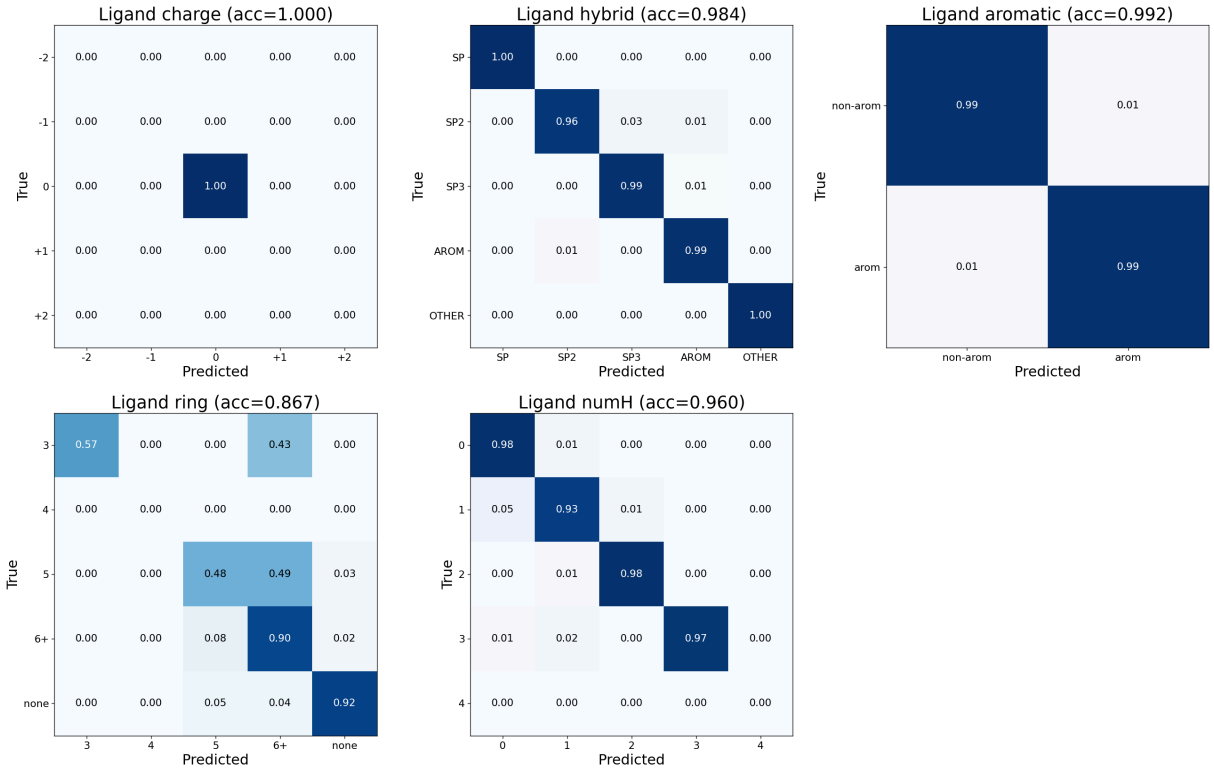
In [ ]:
```
SMALL_VOCAB = 5
HIGH_CONTRAST = 0.5

def plot_confusion(target, pred, vocab, ax, title, *, tick_fontsize=13):
    n = len(vocab)
    cm = np.zeros((n, n), dtype=np.int64)
    for t, p in zip(target, pred, strict=False):
```

```
        if 0 <= t < n and 0 <= p < n:
            cm[t, p] += 1
    cm_norm = cm.astype(float) / cm.sum(axis=1, keepdims=True).clip(min=1)
    ax.imshow(cm_norm, cmap="Blues", vmin=0, vmax=1)
    ax.set_xticks(range(n))
    ax.set_yticks(range(n))
    ax.set_xticklabels(vocab, rotation=90 if n > SMALL_VOCAB else 0, fontsiz
    ax.set_yticklabels(vocab, fontsize=tick_fontsize)
    ax.set_xlabel("Predicted")
    ax.set_ylabel("True")
    ax.set_title(title)
    if n <= SMALL_VOCAB:
        for i in range(n):
            for j in range(n):
                v = cm_norm[i, j]
                ax.text(j, i, f"{v:.2f}", ha="center", va="center",
                        color="white" if v > HIGH_CONTRAST else "black", for

_fig1, _ax1 = plt.subplots(1, 3, figsize=(30, 9))
plot_confusion(
    lig_metrics["categorical"]["element"]["target"],
    lig_metrics["categorical"]["element"]["pred"],
    LIGAND_ELEMENT_VOCAB, _ax1[0],
    f"Ligand element (acc={lig_metrics['categorical']['element']['accuracy']
)
plot_confusion(
    prot_metrics["categorical"]["aa"]["target"],
    prot_metrics["categorical"]["aa"]["pred"],
    PROTEIN_AA_VOCAB, _ax1[1],
    f"Protein aa (acc={prot_metrics['categorical']['aa']['accuracy']:.3f})",
)
plot_confusion(
    prot_metrics["categorical"]["bb_sc"]["target"],
    prot_metrics["categorical"]["bb_sc"]["pred"],
    BB_SC_VOCAB, _ax1[2],
    f"Protein bb_sc (acc={prot_metrics['categorical']['bb_sc']['accuracy']:.
)
_fig1.tight_layout()
_fig1
```



In [ ]:
```
_labels = {
    "charge": ("-2", "-1", "0", "+1", "+2"),
    "hybrid": ("SP", "SP2", "SP3", "AROM", "OTHER"),
    "aromatic": ("non-arom", "arom"),
```

```
    "ring": ("3", "4", "5", "6+", "none"),
    "numH": ("0", "1", "2", "3", "4"),
}
_fig2, _ax2 = plt.subplots(2, 3, figsize=(22, 14))
_flat = _ax2.ravel()
for _a, (_h, _lab) in zip(_flat, _labels.items(), strict=False):
    _m = lig_metrics["categorical"][_h]
    plot_confusion(_m["target"], _m["pred"], _lab, _a,
                   f"Ligand {_h} (acc={_m['accuracy']:.3f})")
_flat[-1].axis("off")
_fig2.tight_layout()
_fig2
```



3. Codebook utilization & protein/ligand sharing

codebook は **1 つ**。protein-atom / ligand-atom が同じコード空間をどう使うか（共有 vs 分離）を見るのが unified 設計の肝。

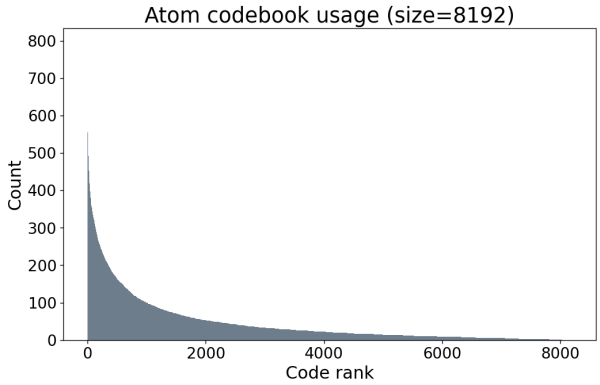
In [ ]:
```
def codebook_stats(indices, codebook_size, name):
    idx = indices.cpu().numpy()
    unique = np.unique(idx)
    counts = np.bincount(idx, minlength=codebook_size).astype(float)
    probs = counts / counts.sum()
    pnz = probs[probs > 0]
    ppl = float(np.exp(-np.sum(pnz * np.log(pnz))))
    print(f"=== {name} (over {len(idx)} atoms) ===")
    print(f"  Active codes : {len(unique)} / {codebook_size} (util {len(unic
    print(f"  Perplexity   : {ppl:.1f}")
    return counts, set(unique.tolist())
```

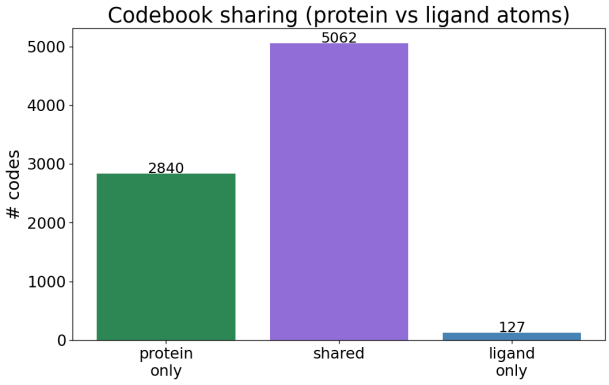
```
cb_size = config.atom.codebook_size
prot_counts, prot_codes = codebook_stats(prot_out["indices"], cb_size, "Prot
lig_counts, lig_codes = codebook_stats(lig_out["indices"], cb_size, "Ligand
all_counts = prot_counts + lig_counts
codebook_stats(
    torch.cat([prot_out["indices"], lig_out["indices"]]), cb_size, "All atom
)
shared = prot_codes & lig_codes
print()
print(f"protein-only codes : {len(prot_codes - lig_codes)}")
print(f"ligand-only codes  : {len(lig_codes - prot_codes)}")
print(f"shared codes       : {len(shared)}")
```

```
=== Protein atoms (over 338810 atoms) ===
  Active codes : 7902 / 8192 (util 0.965)
  Perplexity   : 3785.4
=== Ligand atoms (over 35795 atoms) ===
  Active codes : 5189 / 8192 (util 0.633)
  Perplexity   : 1788.5
=== All atoms (over 374605 atoms) ===
  Active codes : 8029 / 8192 (util 0.980)
  Perplexity   : 4036.0

protein-only codes : 2840
ligand-only codes  : 127
shared codes       : 5062
```

In [ ]:

```
_fig, _ax = plt.subplots(1, 2, figsize=(18, 6))
_ax[0].bar(range(len(all_counts)), np.sort(all_counts)[::-1], width=1.0, col
_ax[0].set_xlabel("Code rank")
_ax[0].set_ylabel("Count")
_ax[0].set_title(f"Atom codebook usage (size={config.atom.codebook_size})")

_po = len(prot_codes - lig_codes)
_lo = len(lig_codes - prot_codes)
_sh = len(shared)
_ax[1].bar(["protein\nonly", "shared", "ligand\nonly"], [_po, _sh, _lo],
           color=["seagreen", "mediumpurple", "steelblue"])
_ax[1].set_ylabel("# codes")
_ax[1].set_title("Codebook sharing (protein vs ligand atoms)")
for _i, _v in enumerate([_po, _sh, _lo]):
    _ax[1].text(_i, _v + 5, str(_v), ha="center", fontsize=15)
_fig.tight_layout()
_fig
```





4. 3D Reconstruction RMSD (Å)

記述子空間ではなく実際の 3D 座標に復元した上での RMSD。pocket は **全 heavy atom**、ligand は heavy atom。両者は同じ pocket canonical frame に載るので per-atom (生) と Kabsch (内部形状) を出す。N_SAMPLES_3D 複合体をサンプル。

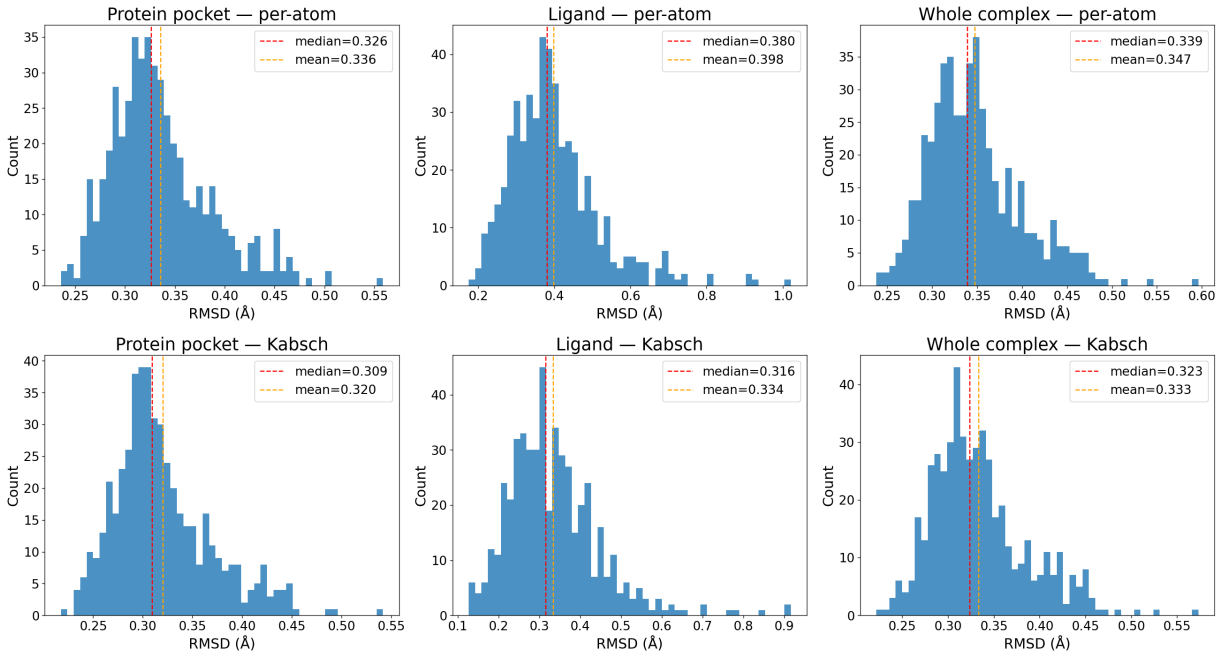
In [ ]:
```
import gzip
import re
import tarfile
from collections import defaultdict
from functools import lru_cache

import pyarrow.parquet as pq

from src.config import PocketExtractionConfig
from src.tokenizers.atom import (
    LigandAtomDescriptor,
    ProteinAtomDescriptor,
    atom_descriptor_to_coords,
    precompute_receptor_atom_features,
)
from src.tokenizers.descriptor_schema import ATOM_DESCRIPTOR_DIM as _ADIM
from src.tokenizers.descriptor_schema import ATOM_LAYOUT as _ALAYOUT
from src.tokenizers.descriptor_schema import fields_by_name as _fbn
from src.tokenizers.ligand import parse_sdf_text
from src.tokenizers.protein import (
    _compute_canonical_frame,
    extract_pocket_atoms_from_candidates,
    precompute_pocket_atom_candidates,
)

def kabsch_align(p, q):
    p_c, q_c = p - p.mean(0), q - q.mean(0)
    u, _, vt = np.linalg.svd(q_c.T @ p_c)
    d = np.sign(np.linalg.det(vt.T @ u.T))
    q_al = q_c @ (vt.T @ np.diag([1.0, 1.0, d]) @ u.T).T
    return p_c, q_al, float(np.sqrt(np.mean(np.sum((p_c - q_al) ** 2, -1))))

pocket_config = PocketExtractionConfig(max_residues=50)
prot_desc_calc = ProteinAtomDescriptor()
lig_desc_calc = LigandAtomDescriptor()
```

```
hub_cache = project_root / "data" / "hub_cache"
receptor_dir = hub_cache / "receptors"
ligand_repo = hub_cache / "repo" / "ligands"

mdf = pq.read_table(hub_cache / "repo" / "manifest.parquet").to_pandas()
test_df = mdf[
    (mdf["source_type"] == "cdonly")
    & (mdf["cdonly_fold0"] == "test")
    & (mdf["label"] == 1)
    & (mdf["ligand_sdf_gz"].str.endswith("_min.sdf.gz"))
].reset_index(drop=True)

mean_t = norm_stats["atom_mean"].to(device)
std_t = norm_stats["atom_std"].to(device)
cf = _fbn(_ALAYOUT)["coord"]

@lru_cache(maxsize=512)
def _receptor(rec_rel):
    path = receptor_dir / rec_rel
    if not path.exists():
        return None
    return precompute_pocket_atom_candidates(path), precompute_receptor_atom

def _encode_decode_coords(desc_np, meta):
    t = torch.from_numpy(desc_np).to(device)
    norm = (t - mean_t) / std_t
    with torch.no_grad():
        idx = vqvae.encode(norm)
        outs = vqvae.decode_to_outputs(idx)
    coord_denorm = outs["coord"] * std_t[cf.start : cf.end] + mean_t[cf.star
    recon = np.zeros((desc_np.shape[0], _ADIM), dtype=np.float32)
    recon[:, cf.start : cf.end] = coord_denorm.cpu().numpy()
    return atom_descriptor_to_coords(recon, meta)

N_SAMPLES_3D = 500
rng = np.random.default_rng(42)
cand = test_df.iloc[rng.choice(len(test_df), min(N_SAMPLES_3D * 5, len(test_
shard_to_pairs = defaultdict(dict)
for row in cand.itertuples(index=False):
    shard_to_pairs[int(row.shard_idx)][int(row.pair_idx)] = f"{row.complex_c
shard_order = list(shard_to_pairs)
rng.shuffle(shard_order)
member_re = re.compile(r"(\d+)\.sdf\.gz$")

def iter_sampled():
    for si in shard_order:
        wanted = shard_to_pairs[si]
        tar_path = ligand_repo / f"{si:06d}.tar"
        if not tar_path.exists():
            continue
        with tarfile.open(tar_path, "r|") as tar:
            for m in tar:
                if not m.isfile():
                    continue
                mt = member_re.search(m.name.rsplit("/", 1)[-1])
                if mt is None:
```

```
                    continue
                rec_rel = wanted.get(int(mt.group(1)))
                if rec_rel is None:
                    continue
                fo = tar.extractfile(m)
                if fo is None:
                    continue
                mols = parse_sdf_text(gzip.decompress(fo.read()).decode("utf
                if mols:
                    yield rec_rel, mols[0]

prot_pa, prot_kb, lig_pa, lig_kb, joint_pa, joint_kb = [], [], [], [], [], [
n_done = 0
for rec_rel, mol in iter_sampled():
    if n_done >= N_SAMPLES_3D:
        break
    rec = _receptor(rec_rel)
    if rec is None:
        continue
    precomp, feats = rec
    heavy = np.array([(a[1], a[2], a[3]) for a in mol["atoms"] if a[0] != "H
    if len(heavy) == 0:
        continue
    pocket = extract_pocket_atoms_from_candidates(precomp, heavy, pocket_cor
    if pocket is None or pocket.atom_coords.shape[0] == 0:
        continue
    centroid, rotation = _compute_canonical_frame(pocket.ca_coords.astype(np
    frame = (centroid, rotation)

    pdesc, pmeta = prot_desc_calc.compute(pocket, feats, frame)
    prot_recon = _encode_decode_coords(pdesc, pmeta)
    prot_orig = pocket.atom_coords.astype(np.float64)

    ldesc, _e, lmeta = lig_desc_calc.compute(mol["atoms"], mol["bonds"], fra
    if len(ldesc) == 0:
        continue
    lig_recon = _encode_decode_coords(ldesc, lmeta)
    lig_orig = np.array(
        [(mol["atoms"][i][1], mol["atoms"][i][2], mol["atoms"][i][3]) for i
        dtype=np.float64,
    )

    prot_pa.append(float(np.sqrt(np.mean(np.sum((prot_orig - prot_recon) **
    prot_kb.append(kabsch_align(prot_orig, prot_recon)[2])
    lig_pa.append(float(np.sqrt(np.mean(np.sum((lig_orig - lig_recon) ** 2,
    lig_kb.append(kabsch_align(lig_orig, lig_recon)[2])
    jo, jr = np.vstack([prot_orig, lig_orig]), np.vstack([prot_recon, lig_re
    joint_pa.append(float(np.sqrt(np.mean(np.sum((jo - jr) ** 2, -1)))))
    joint_kb.append(kabsch_align(jo, jr)[2])
    n_done += 1

prot_pa, prot_kb = np.array(prot_pa), np.array(prot_kb)
lig_pa, lig_kb = np.array(lig_pa), np.array(lig_kb)
joint_pa, joint_kb = np.array(joint_pa), np.array(joint_kb)
print(f"Evaluated {len(prot_pa)} complexes (all-atom 3D RMSD)\n")
for _n, _a in [
```

```
    ("Protein pocket — per-atom", prot_pa), ("Protein pocket — Kabsch", prot
    ("Ligand — per-atom", lig_pa), ("Ligand — Kabsch", lig_kb),
    ("Whole complex — per-atom", joint_pa), ("Whole complex — Kabsch", joint
]:
    print(f"{_n}: mean {_a.mean():.4f}  median {np.median(_a):.4f}  std {_a.

def plot_rmsd_hist(ax, arr, title):
    ax.hist(arr, bins=50, alpha=0.8)
    ax.axvline(np.median(arr), color="r", linestyle="--", label=f"median={np
    ax.axvline(arr.mean(), color="orange", linestyle="--", label=f"mean={arr
    ax.set_xlabel("RMSD (Å)")
    ax.set_ylabel("Count")
    ax.set_title(title)
    ax.legend()

_fig, _ax = plt.subplots(2, 3, figsize=(22, 12))
plot_rmsd_hist(_ax[0, 0], prot_pa, "Protein pocket — per-atom")
plot_rmsd_hist(_ax[0, 1], lig_pa, "Ligand — per-atom")
plot_rmsd_hist(_ax[0, 2], joint_pa, "Whole complex — per-atom")
plot_rmsd_hist(_ax[1, 0], prot_kb, "Protein pocket — Kabsch")
plot_rmsd_hist(_ax[1, 1], lig_kb, "Ligand — Kabsch")
plot_rmsd_hist(_ax[1, 2], joint_kb, "Whole complex — Kabsch")
_fig.tight_layout()
_fig
```

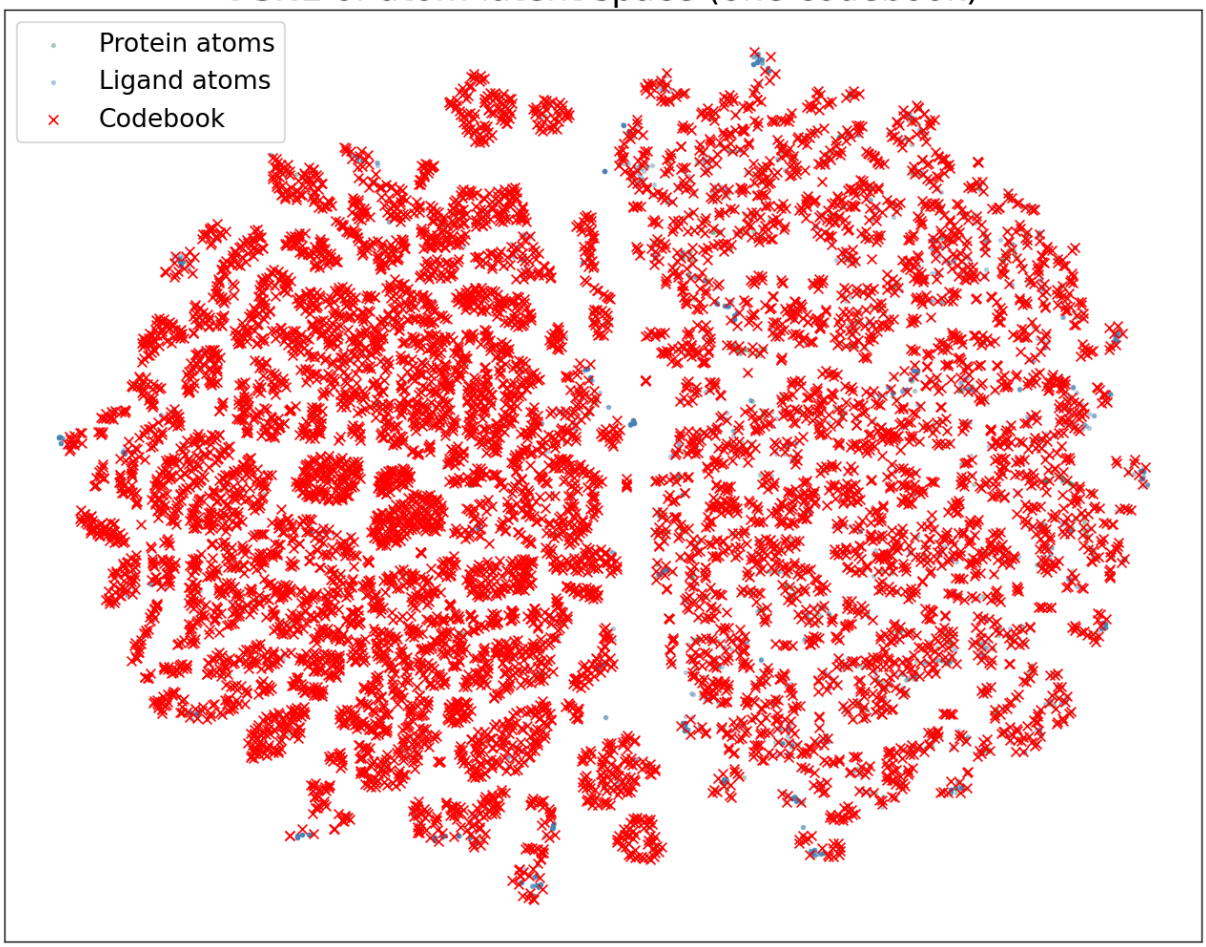
```
[13:20:23] Explicit valence for atom # 149 O, 3, is greater than permitted
[13:20:25] Explicit valence for atom # 13 N, 4, is greater than permitted
[13:20:30] Explicit valence for atom # 12 N, 4, is greater than permitted
[13:20:49] Explicit valence for atom # 15 N, 4, is greater than permitted
[13:21:01] Explicit valence for atom # 2271 O, 3, is greater than permitted
[13:21:04] Explicit valence for atom # 1 C, 5, is greater than permitted
[13:21:22] Explicit valence for atom # 17 N, 4, is greater than permitted
[13:21:24] Explicit valence for atom # 18 N, 4, is greater than permitted
[13:21:26] Explicit valence for atom # 8 N, 4, is greater than permitted
[13:21:27] Explicit valence for atom # 8 N, 4, is greater than permitted
[13:21:28] Explicit valence for atom # 11 N, 4, is greater than permitted
[13:21:36] Explicit valence for atom # 133 O, 3, is greater than permitted
Evaluated 500 complexes (all-atom 3D RMSD)

Protein pocket — per-atom: mean 0.3358  median 0.3264  std 0.0501
Protein pocket — Kabsch: mean 0.3199  median 0.3095  std 0.0492
Ligand — per-atom: mean 0.3981  median 0.3803  std 0.1188
Ligand — Kabsch: mean 0.3340  median 0.3155  std 0.1124
Whole complex — per-atom: mean 0.3474  median 0.3393  std 0.0519
Whole complex — Kabsch: mean 0.3335  median 0.3234  std 0.0517
```



5. Latent space (t-SNE)

エンコーダ出力（protein / ligand 別に色分け）と codebook ベクトルを 2D 射影。
protein/ligand が共有空間でどう分布するかを見る。

In [ ]:
```
from sklearn.manifold import TSNE

rng_t = np.random.default_rng(42)
n_each = 2500
pz = prot_out["z"].cpu().numpy()
lz = lig_out["z"].cpu().numpy()
pi = rng_t.choice(len(pz), min(n_each, len(pz)), replace=False)
li = rng_t.choice(len(lz), min(n_each, len(lz)), replace=False)
cb_vecs = vqvae.codebook.embedding.cpu().detach().numpy()
combined = np.vstack([pz[pi], lz[li], cb_vecs])
emb = TSNE(n_components=2, random_state=42, perplexity=30).fit_transform(com
np_, nl_ = len(pi), len(li)
p_emb, l_emb, cb_emb = emb[:np_], emb[np_ : np_ + nl_], emb[np_ + nl_ :]

_fig, _ax = plt.subplots(figsize=(11, 9))
_ax.scatter(p_emb[:, 0], p_emb[:, 1], s=5, alpha=0.3, c="seagreen", label="P
_ax.scatter(l_emb[:, 0], l_emb[:, 1], s=5, alpha=0.3, c="steelblue", label="
_ax.scatter(cb_emb[:, 0], cb_emb[:, 1], s=35, c="red", marker="x", linewidth
_ax.set_title("t-SNE of atom latent space (one codebook)")
_ax.legend()
_ax.set_xticks([])
_ax.set_yticks([])
_fig.tight_layout()
_fig
```



6. Summary table

In [ ]:
```
import pandas as pd

summary = pd.DataFrame(
    {
        "Metric": [
            "Codebook size",
            "Latent dim",
            "Coord RMSE per-atom (Å)",
            "Element accuracy",
            "aa accuracy",
            "bb_sc accuracy",
            "3D per-atom RMSD (Å)",
            "3D Kabsch RMSD (Å)",
        ],
        "Protein atoms": [
            config.atom.codebook_size,
            config.atom.latent_dim,
            f"{prot_metrics['coord_rmse']:.4f}",
            f"{prot_metrics['categorical']['element']['accuracy']:.4f}",
            f"{prot_metrics['categorical']['aa']['accuracy']:.4f}",
            f"{prot_metrics['categorical']['bb_sc']['accuracy']:.4f}",
            f"{prot_pa.mean():.4f}",
```

```
            f"{prot_kb.mean():.4f}",
        ],
        "Ligand atoms": [
            config.atom.codebook_size,
            config.atom.latent_dim,
            f"{lig_metrics['coord_rmse']:.4f}",
            f"{lig_metrics['categorical']['element']['accuracy']:.4f}",
            "— (n/a)",
            "— (n/a)",
            f"{lig_pa.mean():.4f}",
            f"{lig_kb.mean():.4f}",
        ],
    }
)
summary
```

	Metric	Protein atoms	Ligand atoms
0	Codebook size	8192	8192
1	Latent dim	16	16
2	Coord RMSE per-atom (Å)	0.3354	0.4426
3	Element accuracy	0.9999	0.9972
4	aa accuracy	0.9996	— (n/a)
5	bb_sc accuracy	0.9998	— (n/a)
6	3D per-atom RMSD (Å)	0.3358	0.3981
7	3D Kabsch RMSD (Å)	0.3199	0.3340